5

STATE-SPACE SEARCH

Optimal Search

Optimal Informed Search: **A* Search**

n.h.reyes@massey.ac.nz

Classes of Search

Class	Name	Operation
Any Path Uninformed	**Depth-First**	Systematic exploration of the whole tree until a goal is found
	Breadth-First	
Any Path Informed	**Best-First**	Uses heuristic measure of goodness of a state (e.g. estimated distance to goal)
Optimal Uninformed	**Uniform-Cost**	Uses path **"length"** measure. Finds "shortest" path.
Optimal Informed	**A***	Uses path **"length"** measure and **heuristic**. Finds "shortest" path.

back

Goal Direction

- **UC** is really trying to identify the **shortest path** **to** **every** **state** in the graph in order. It has **no particular bias** to finding a path to a goal early in the search.

Goal Direction

- **UC** is really trying to identify the shortest path to every state in the graph in order. It has no particular bias to finding a path to a goal early in the search.

- We can introduce such a **bias** by means of heuristic function **h(state(N))**, which is an estimate (**h**) of the distance from a state to the goal.

Goal Direction

- **UC** is really trying to identify the shortest path to every state in the graph in order. It has no particular bias to finding a path to a goal early in the search.

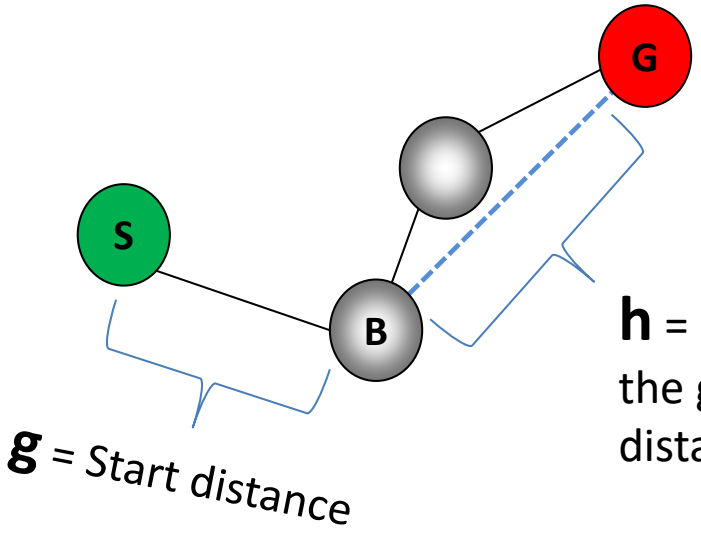
- We can introduce such a bias by means of heuristic function h(state(N)), which is an estimate (h) of the distance from a state to the goal.

- Instead of enumerating paths in order of just length (**g**), enumerate paths in terms of **f = estimated total path length = g + h.**

SEARCH

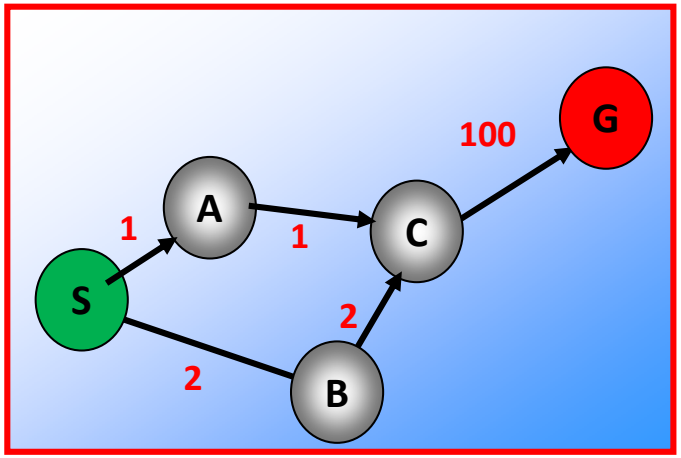
f= g + h

f(BS) = g(S→B) + h(state(BS))



h = Estimate of the goal distance

g = *Start distance*



Heuristic Values:
A=100, B=88, C=100, S=90, G=0

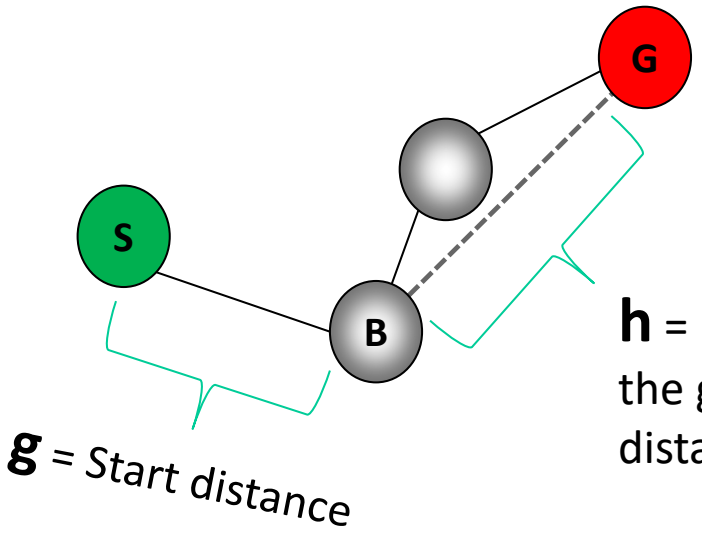
- Instead of enumerating paths in order of just length (**g**), enumerate paths in terms of **f = estimated total path length = g + h.**

- An estimate that always underestimates the real path length to the goal is called **admissible**. For example, an estimate of **0** is **admissible** (**but useless**). Straight line distance is admissible estimate for path length in Euclidean space.

Goal Direction

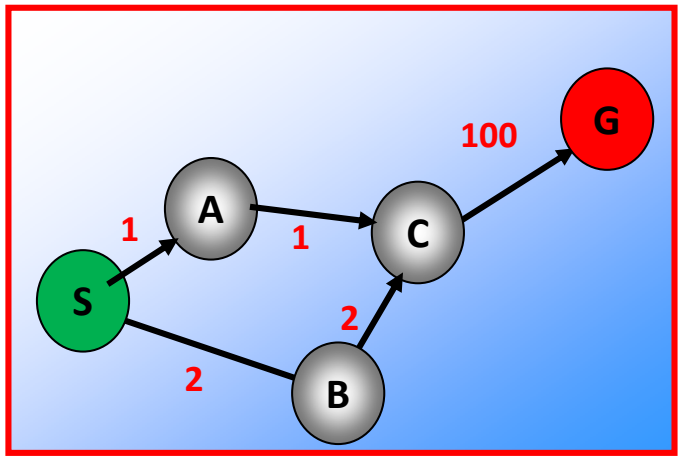
f= g + h

$$f(BS) = g(S \rightarrow B) + h(state(BS))$$



g = *Start distance*

h = Estimate of the goal distance



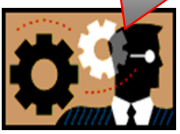
Heuristic Values:
A=100, B=88, C=100, S=90, G=0

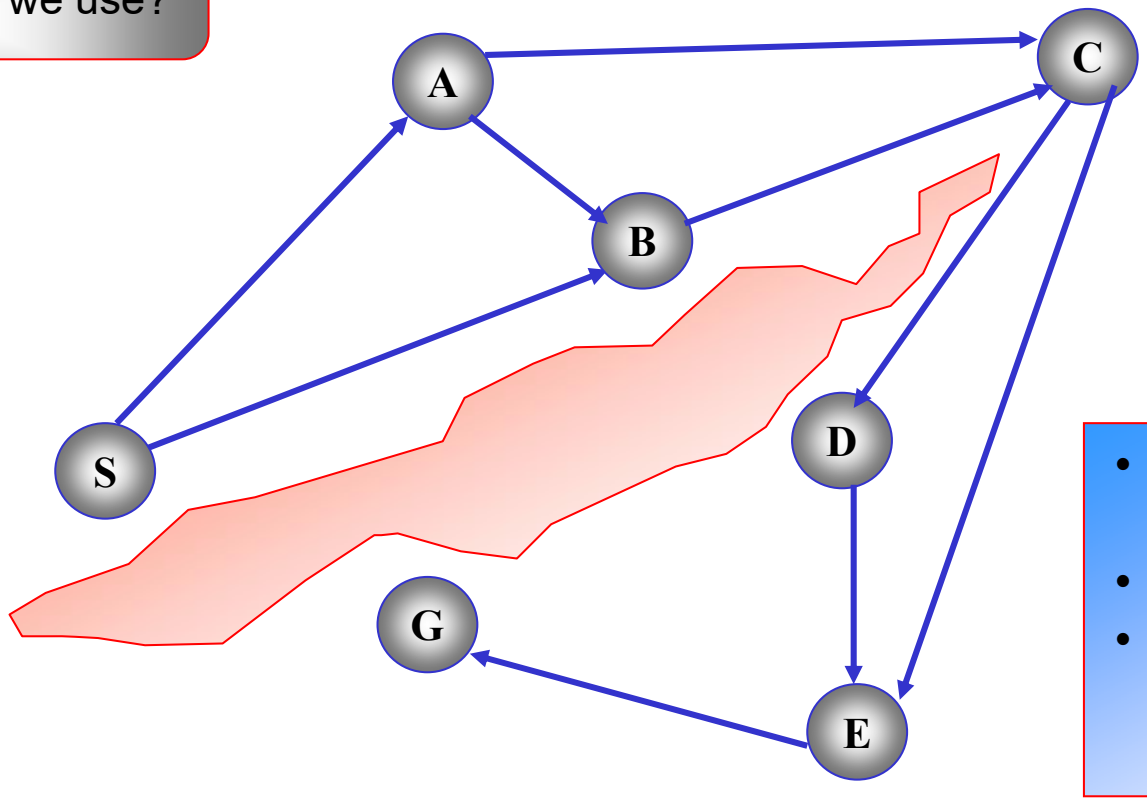
- Use of an **admissible estimate** guarantees that **UC** will still find the **shortest** path.

- **UC with an admissible estimate** is known as **A* Search**.

Straight-Line Estimate

Which heuristic function can we use?



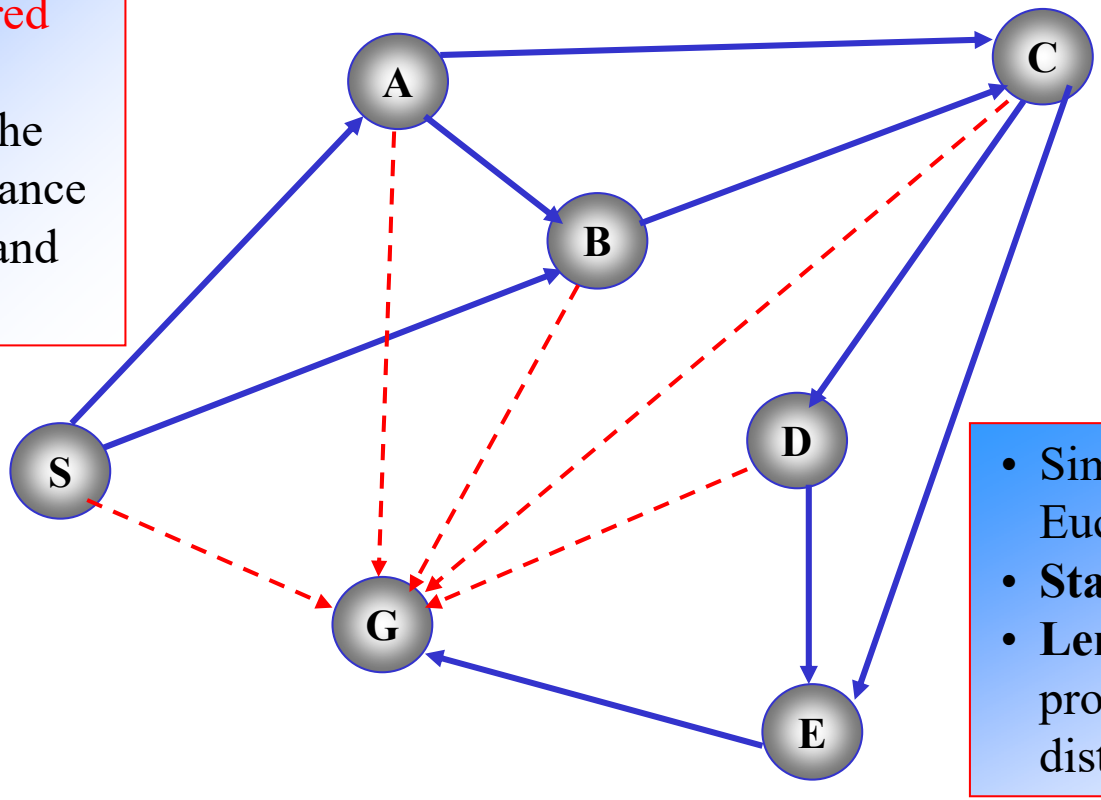


- Simple graph embedded in Euclidean space
- **States** as city locations
- **Length of the links** are proportional to the driving distances between cities

In Euclidean space

Straight-Line Estimate

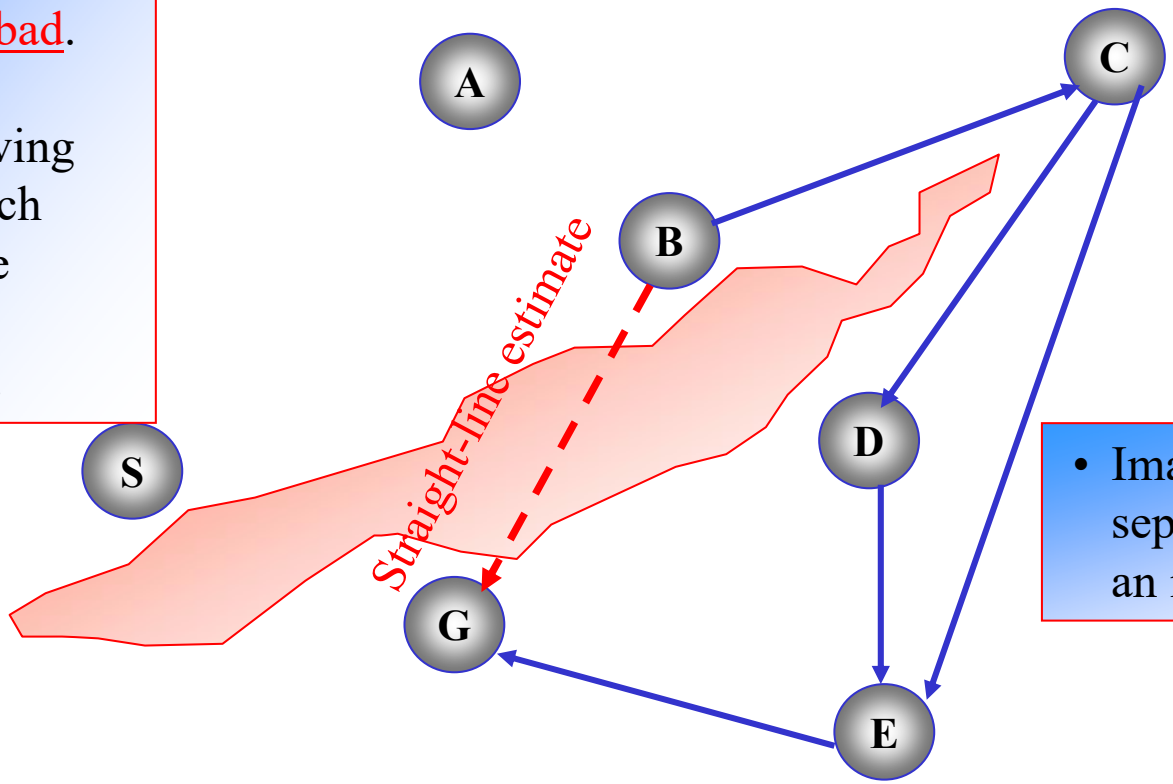
- We can use the straight-line (airline) distances (dotted red lines) as an underestimate of the actual driving distance between any city and the goal.



- Simple graph embedded in Euclidean space
- **States** as city locations
- **Length of the links** are proportional to the driving distances between cities

Straight-Line Estimate

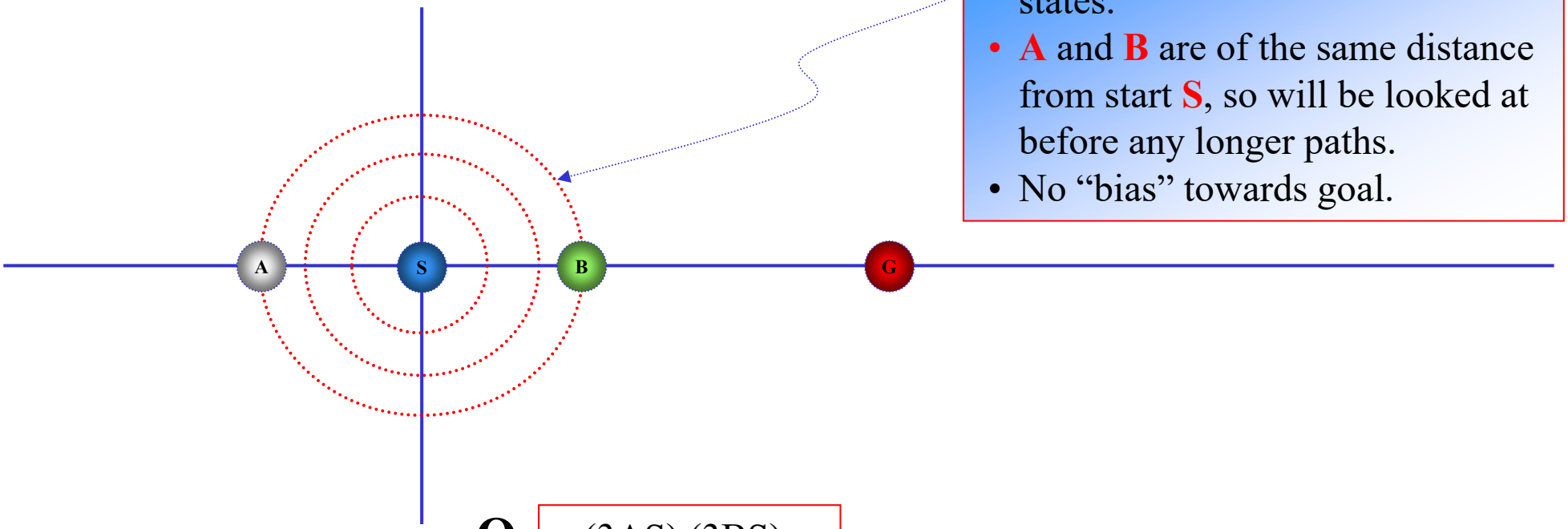
- Here we can see that the straight-line estimate between **B** and **G** is very bad.
- The actual driving distance is much longer than the straight-line underestimate.



- Imagine that **B** and **G** could be separated by a big mountain or an island

Why use estimate of goal distance?

- Order in which **UC** looks at states.
- **A** and **B** are of the same distance from start **S**, so will be looked at before any longer paths.
- No "bias" towards goal.



i.e. **Q** (3AS) (3BS)

- Assume **states** are **points** in the Euclidean space

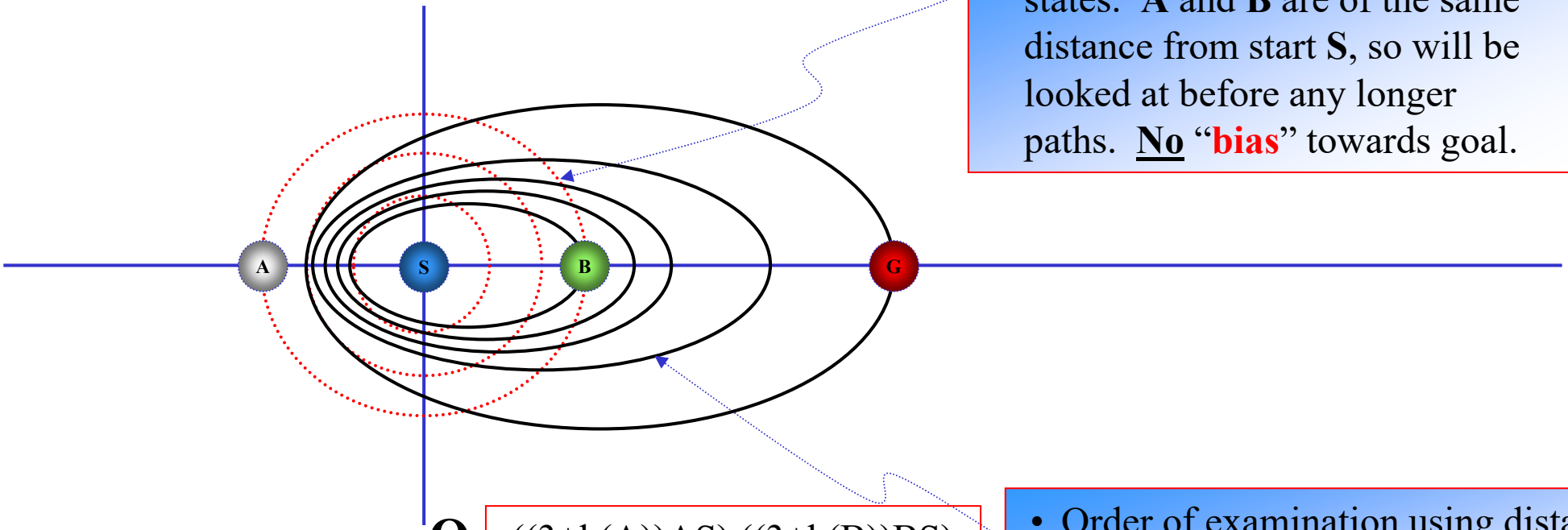
Based on the ordering, this node is will be expanded first

SEARCH

Why use estimate of goal distance?

- Assume **states** are **points** in the Euclidean space

- Order in which **UC** looks at states. **A** and **B** are of the same distance from start **S**, so will be looked at before any longer paths. **No** "**bias**" towards goal.



e.g. **Q** ((3+h(A))AS) ((3+h(B))BS)

12 8

Q = (15AS)(11BS)

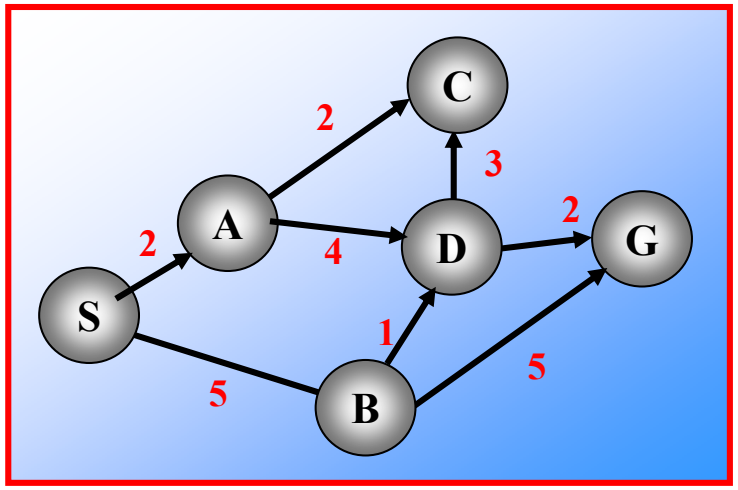
- Order of examination using distance from **S** + estimate of distance to **G**.
- Note: **"bias"** towards the goal;
- The points away from **G** look worse

f= g + h

back

A*

Given the following graph and heuristic values, find a path from **S** to **G** using the A* algorithm.



Heuristic Values:
A=2, B=3, C=1, D=1, S=0, G=0

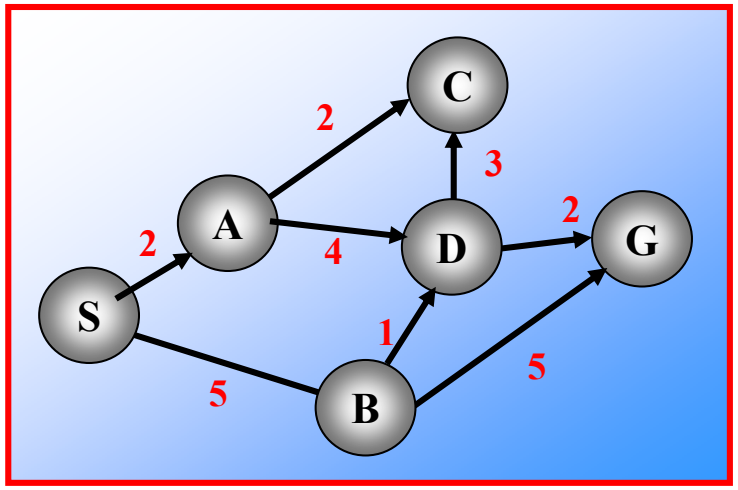
For our convention in class, add the path extensions to the **front** of the Q.



- Pick **best** (**by path length + heuristic**) element of Q, Add path extensions **anywhere** in Q.

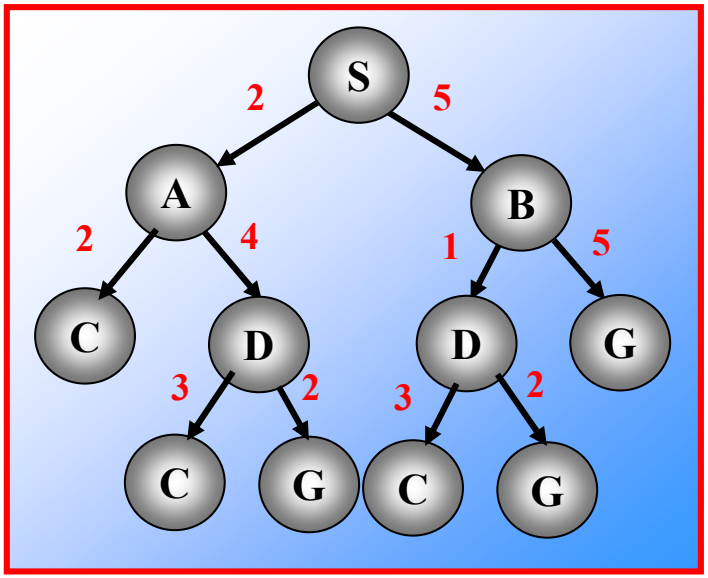
A*

Given the following graph and heuristic values, find a path from **S** to **G** using the A* algorithm.



Heuristic Values:
A=2, B=3, C=1, D=1, S=0, G=0

Search Tree



For our convention in class, add the path extensions to the **front** of the Q.

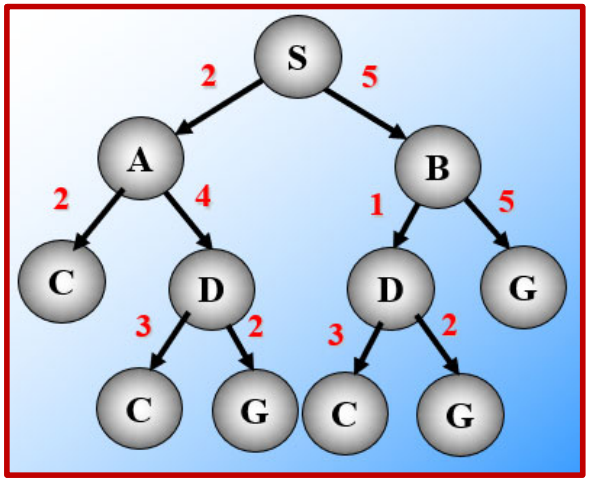


- Pick **best** (**by path length + heuristic**) element of Q, Add path extensions **anywhere** in Q.

SEARCH

A*

Step	Dequeued	Q
0		(0 S)
1	(0 S)	(4 AS)(8 BS)



Heuristic Values:
A=2, B=3, C=1, D=1, S=0, G=0

f= g + h

$f(AS) = g(S \rightarrow A) + h(state(AS))$

$f(AS) = 2 + h(A)$

$= 2 + 2 = 4$

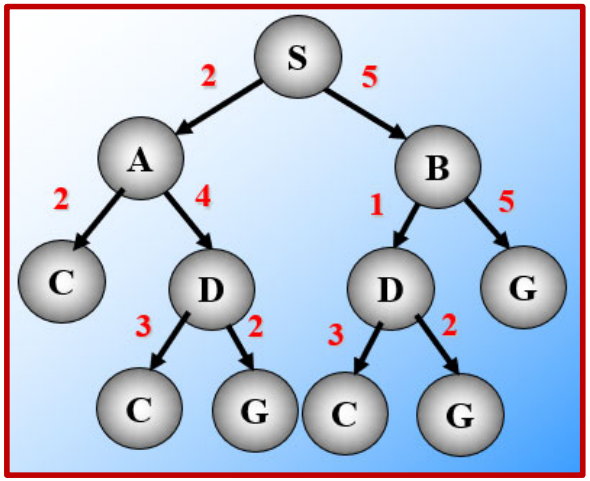
For our convention in class, add the path extensions to the **front** of the Q.

- Pick **best** (**by path length + heuristic**) element of Q, Add path extensions **anywhere** in Q.

SEARCH

A*

Step	Dequeued	Q
0		(0 S)
1	(0 S)	(4 AS)(8 BS)



Heuristic Values:
A=2, B=3, C=1, D=1, S=0, G=0

f= g + h

f(BS) = g(S→B) + h(state(BS))

f(AS) = 5 + h(B)

= 5 + 3 = 8

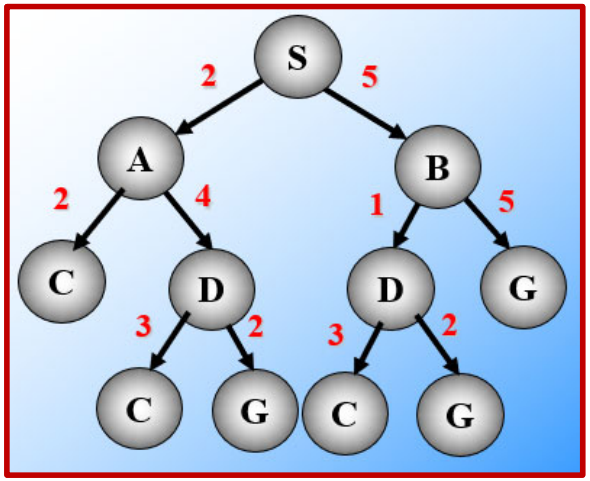
For our convention in class, add the path extensions to the **front** of the Q.

- Pick **best** (**by path length + heuristic**) element of Q, Add path extensions **anywhere** in Q.

SEARCH

A*

Step	Dequeued	Q
0		~~(0 S)~~
1	(0 S)	~~(4 AS)~~(8 BS)
2	(4 AS)	(5 CAS)(7 DAS)(8 BS)



Heuristic Values:
A=2, B=3, C=1, D=1, S=0, G=0

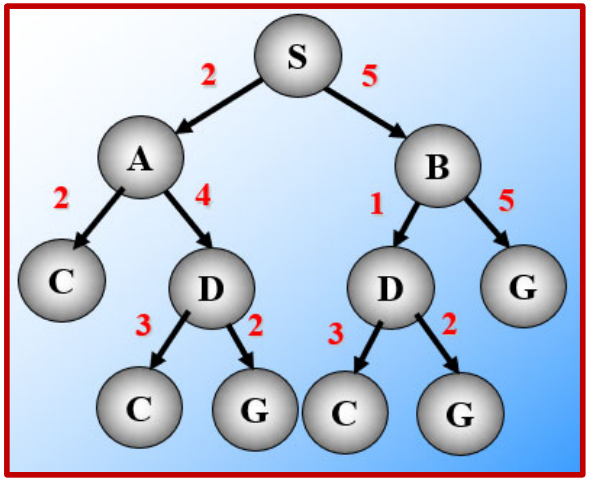
For our convention in class, add the path extensions to the front of the Queue.

- Pick **best** (**by path length + heuristic**) element of Q, Add path extensions **anywhere** in Q.

SEARCH

A*

Step	Dequeued	Q
0		~~(0 S)~~
1	(0 S)	~~(4 AS)~~(8 BS)
2	(4 AS)	(5 CAS)(7 DAS)(8 BS)



Heuristic Values:
A=2, B=3, C=1, D=1, S=0, G=0

f= g + h

$$f(CAS) = g(S \rightarrow A \rightarrow C) + h(state(CAS))$$

$$f(AS) = 4 + h(C)$$
$$= 4 + 1 = 5$$

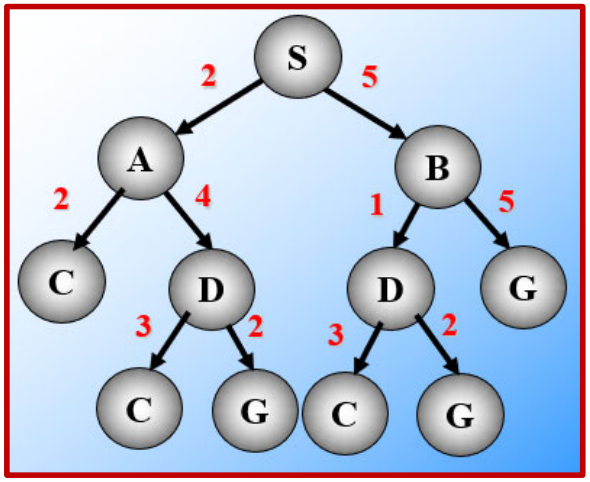
For our convention in class, add the path extensions to the **front** of the Queue.

- Pick **best** (**by path length + heuristic**) element of Q, Add path extensions **anywhere** in Q.

SEARCH

A*

Step	Dequeued	Q
0		~~(0 S)~~
1	(0 S)	(4 ~~AS~~)(8 BS)
2	(4 AS)	(5 ~~CAS~~)(7 DAS)(8 BS)
3	(5 CAS)	(7 ~~DAS~~)(8 BS)
4	(7 DAS)	(10 CDAS)(8 ~~GDAS~~) (8 BS)
5	(8 GDAS)	



Heuristic Values:
A=2, B=3, C=1, D=1, S=0, G=0

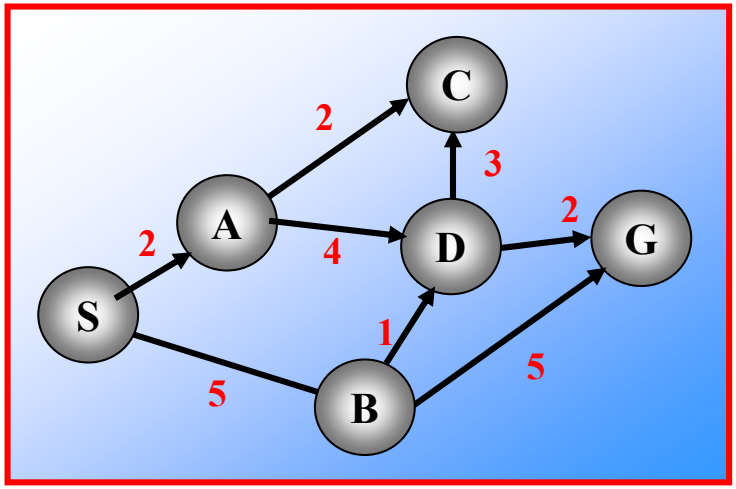
Sequence of State Expansions: S – A – C – D - G

For our convention in class, add the path extensions to the **front** of the Queue.

- Pick **best** (**by path length + heuristic**) element of Q, Add path extensions **anywhere** in Q.

Not all heuristics are admissible

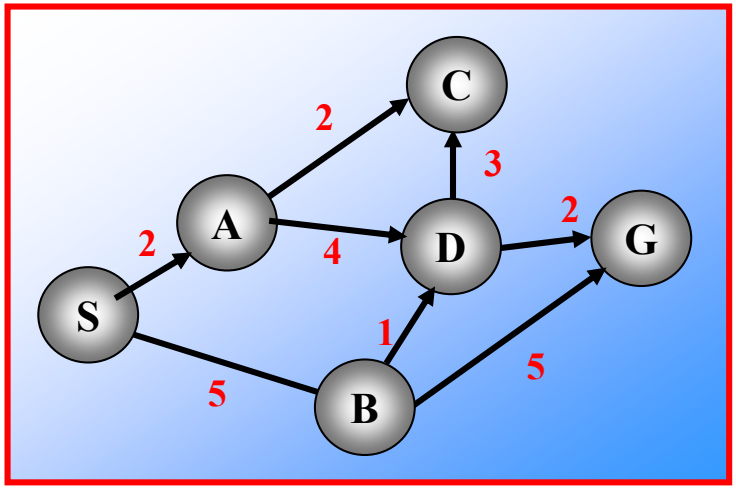
Given the link lengths in the figure, is the table of heuristic values that we used in our earlier Best-First example an **admissible heuristic**?



Heuristic values:
A=2, B=3, C=1, D=4, S=10, G=0

Not all heuristics are admissible

Given the link lengths in the figure, is the table of heuristic values that we used in our earlier Best-First example an **admissible heuristic**?



Heuristic values:
A=2, B=3, C=1, D=4, S=10, G=0

No!

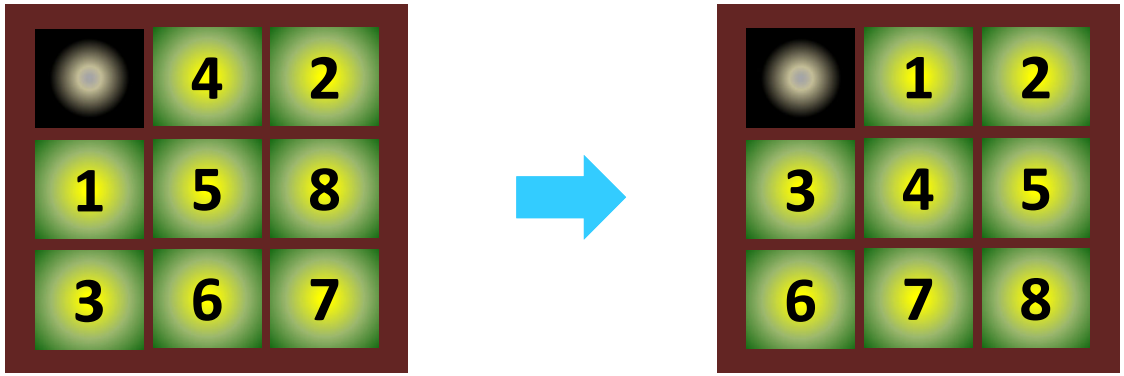
- A is ok
- B is ok
- C is ok
- **D** is **too big**, needs to be <= 2
- S is too big, can always use 0 for start

Solving the 8-Puzzle using Search

back

8 Puzzle Problem

Given the **start** state, find the sequence of moves that will arrange the tiles according to the **goal** state.



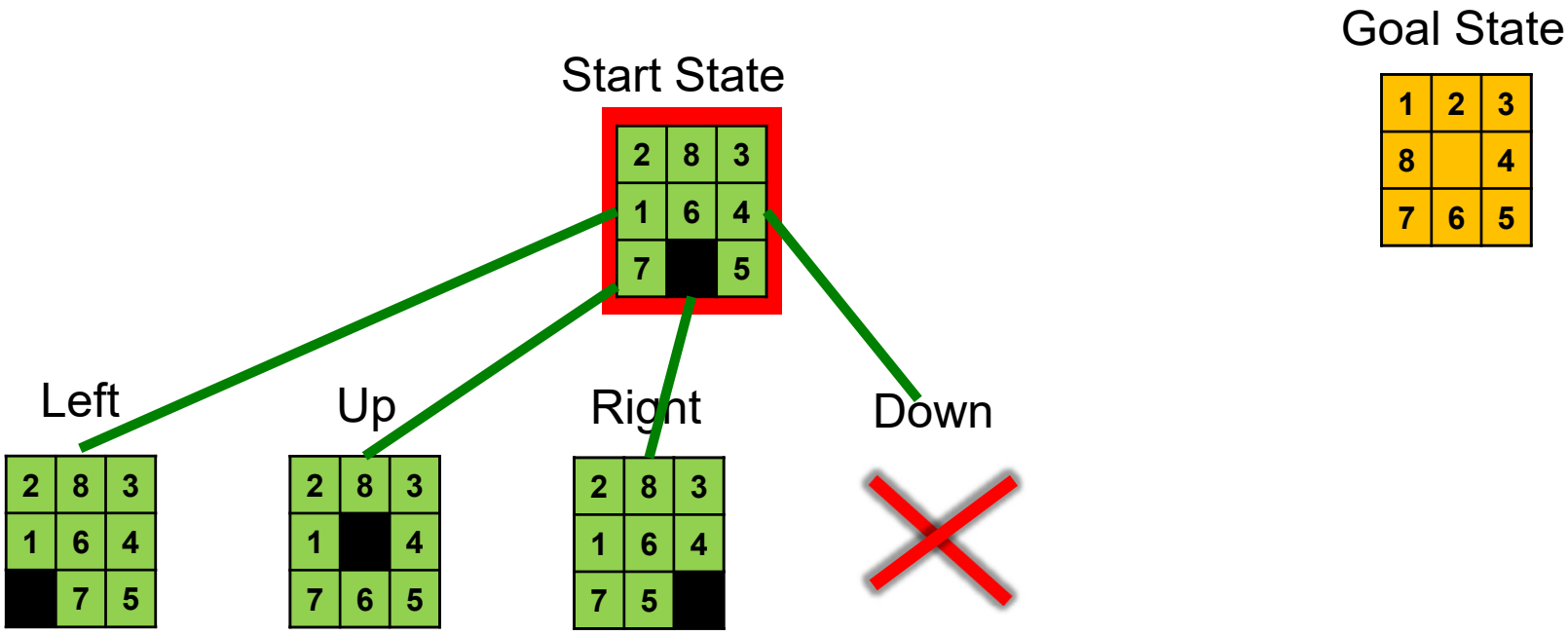
Start **Goal**

Tiles can be moved **up, down, left,** or **right**, provided that the following conditions are met:

- There is no other tile blocking the direction of the movement.
- Tiles are not moved outside of the boundaries.

- Let’s look at the details of how A* solves this problem.

Search

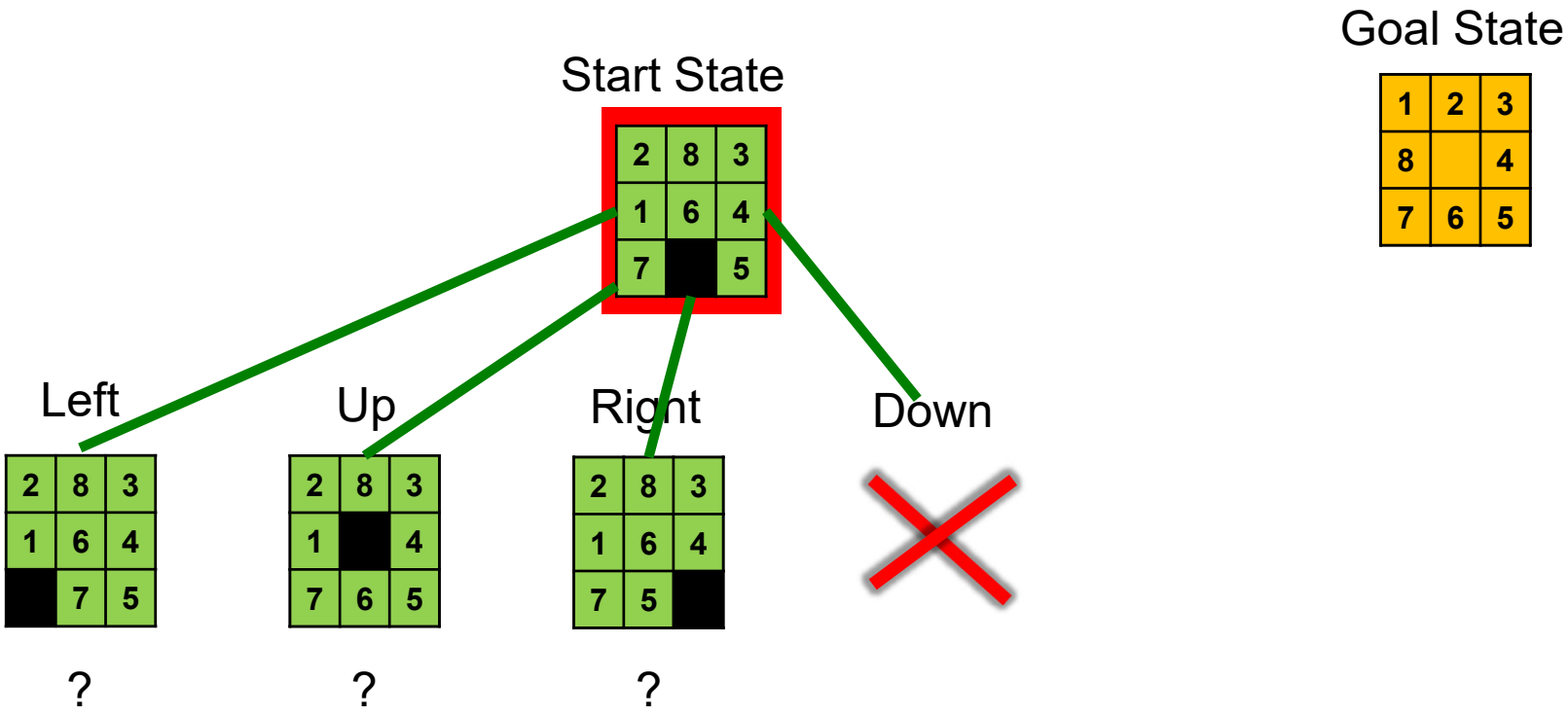


Given the **start state**, we can examine the different possible moves that can be carried out from that state.

Think of the moves as moving the **blank tile** in the following order: Left, Up, Right, Down

menu

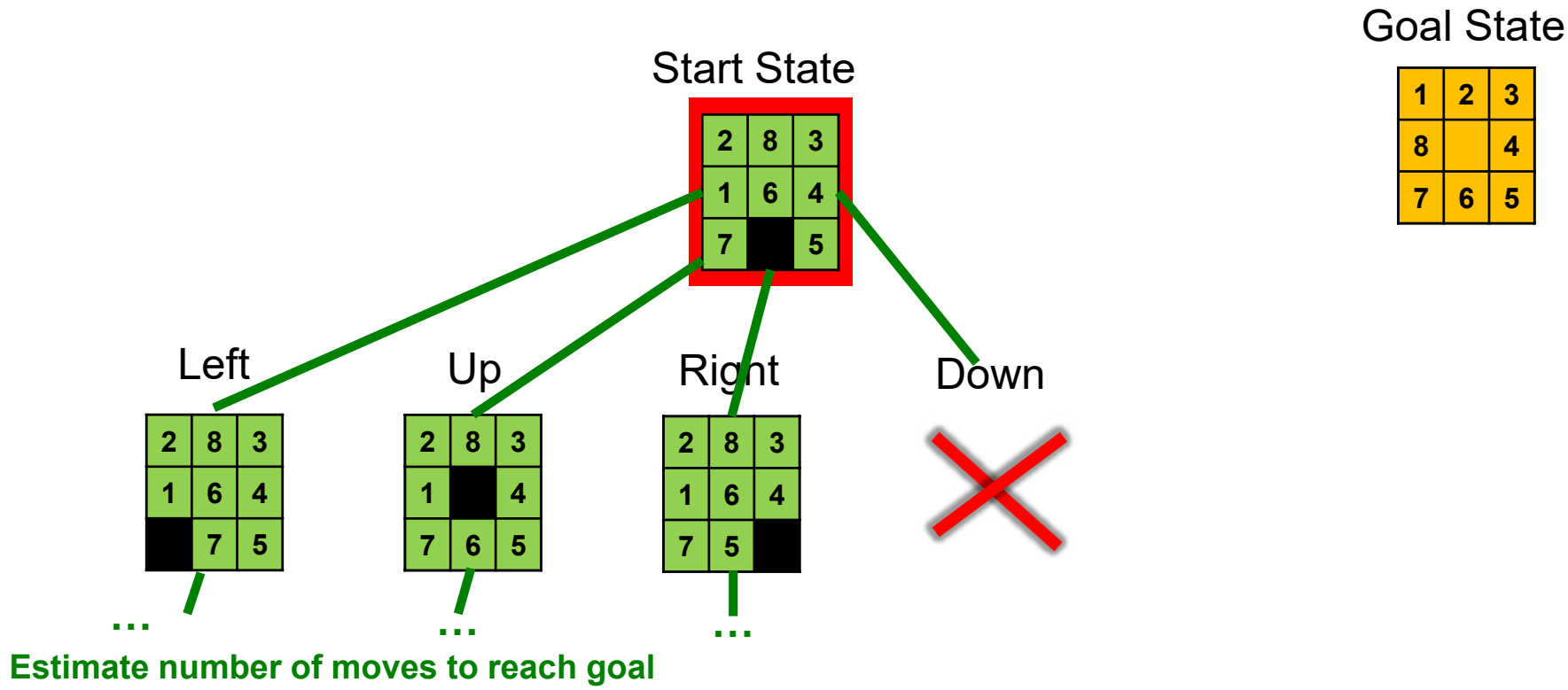
Search



But can we tell which among the configuration of tiles will likely lead us to the goal more quickly?

menu

Heuristic Search



But can we tell which among the config. of tiles will likely lead us to the goal more quickly?

We can use a heuristic function to find an estimate of the remaining number of moves to reach the goal.

Sum of Manhattan Distance of tile to its goal location

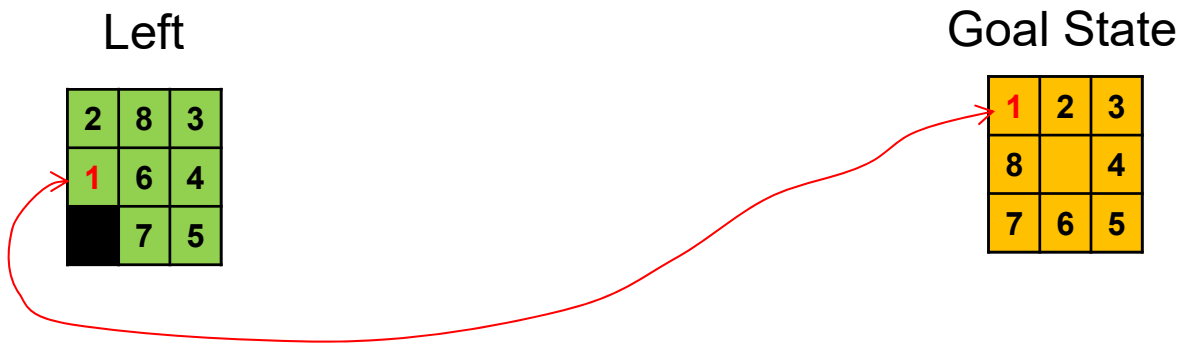
Manhattan Distance between (x_1, y_1) and (x_2, y_2) is $|x_1 - x_2|$ and $|y_1 - y_2|$.

Each move can only decrease the distance of exactly one tile.

menu

Heuristic Search

Let's apply the Manhattan Distance formula to the following puzzle configuration.



Tile 1 *is 1 distance unit away from its target position*

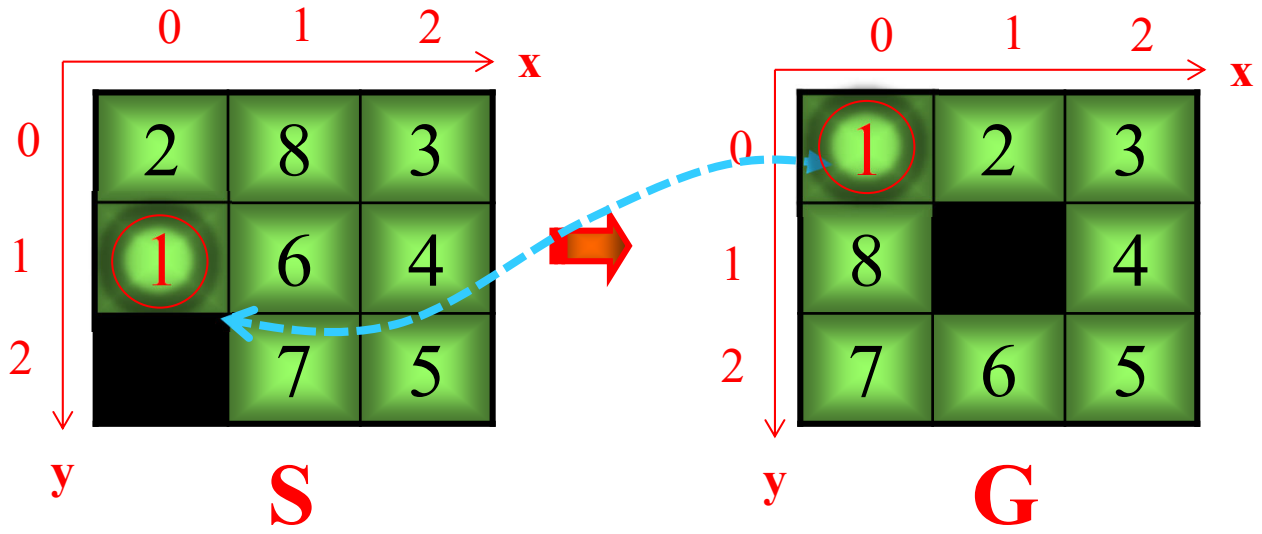
Sum of Manhattan Distance of tile to its goal location

Manhattan Distance between (x_1, y_1) and (x_2, y_2) is $|x_1 - x_2|$ and $|y_1 - y_2|$.

Each move can only decrease the distance of exactly one tile.

menu

Admissible Heuristics



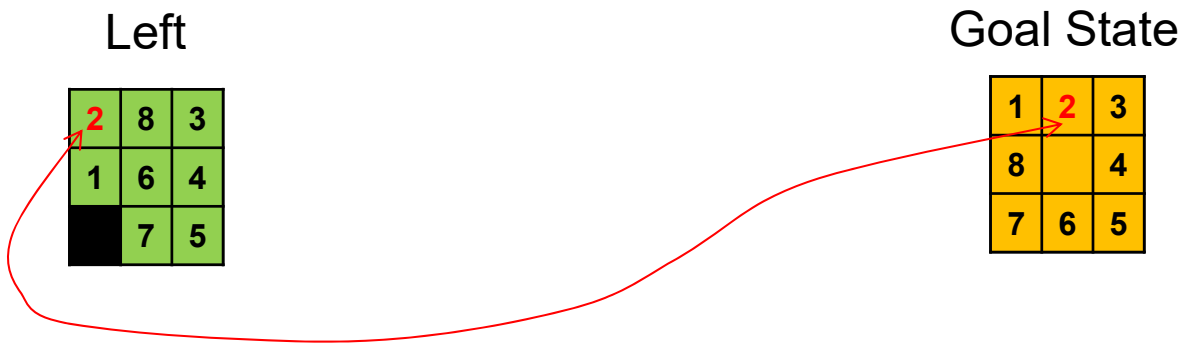
Sample Computations:

Tile 1: Coordinates of Tile 1 in State **S**: (x=0, y=1)
Coordinates of Tile 1 in State **G**: (x=0, y=0)
Manhattan distance = |0-0| + |1-0| = **1**

Manhattan Distance between (x_1, y_1) and $(x_2, y_2) = |x_1 - x_2| + |y_1 - y_2|$

Absolute value

Heuristic Search



Tile 1 *is 1 distance unit away from its target position*

Tile 2 *is 1 distance unit away from its target position*

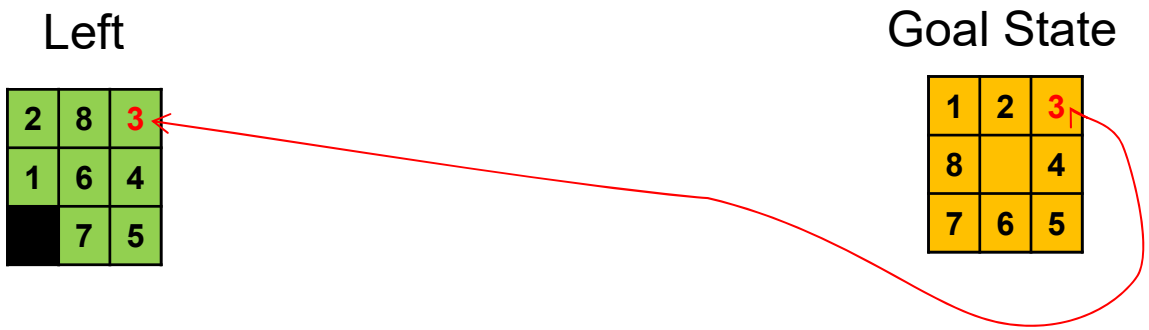
Sum of Manhattan Distance of tile to its goal location

Manhattan Distance between (x_1, y_1) and (x_2, y_2) is $|x_1 - x_2|$ and $|y_1 - y_2|$.

Each move can only decrease the distance of exactly one tile.

menu

Heuristic Search



Tile 1 *is 1 distance unit away from its target position*

Tile 2 *is 1 distance unit away from its target position*

Tile 3 *is 0 distance unit away from its target position*

Sum of Manhattan Distance of tile to its goal location

Manhattan Distance between (x_1, y_1) and (x_2, y_2) is $|x_1 - x_2|$ and $|y_1 - y_2|$.

Each move can only decrease the distance of exactly one tile.

menu

Heuristic Search

Left

2	8	3
1	6	4
	7	5

Goal State

1	2	3
8		4
7	6	5

6

Tile 1 *is 1 distance unit away from its target position*

Tile 2 *is 1 distance unit away from its target position*

Tile 3 *is 0 distance unit away from its target position*

Tile 4 *is 0 distance unit away from its target position*

Tile 5 *is 0 distance unit away from its target position*

Tile 6 *is 1 distance unit away from its target position*

Tile 7 *is 1 distance unit away from its target position*

Tile 8 *is 2 distance unit2 away from its target position*

Sum = 6

Sum of Manhattan Distance of tile to its goal location

Manhattan Distance between (x_1, y_1) and (x_2, y_2) is $|x_1 - x_2|$ and $|y_1 - y_2|$.

Each move can only decrease the distance of exactly one tile.

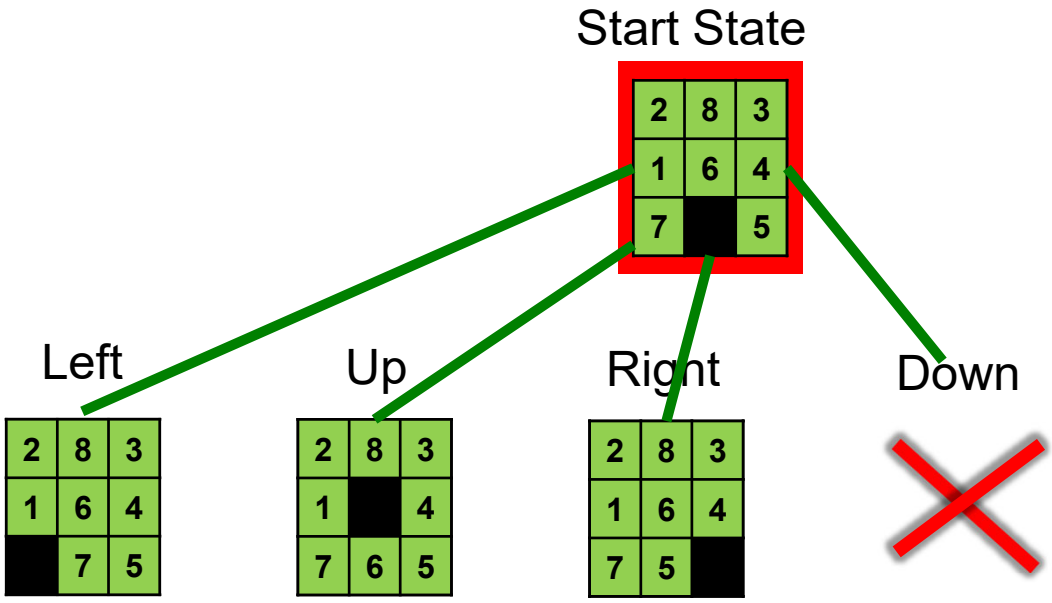
menu

Heuristic Search

Computing for the heuristic values of each of the children of the Start state:

Goal State

1	2	3
8		4
7	6	5

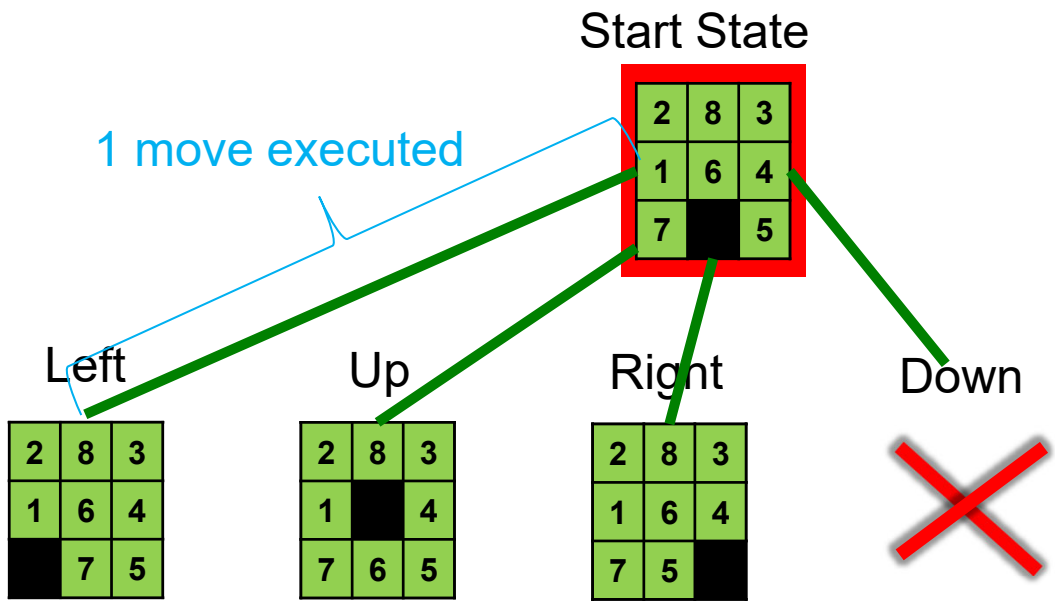


Heuristic value: 6 4 6 ——— **Sum of Manhattan Distance**

menu

Heuristic Search

Next, computing for the **F-cost** of each of the children of the Start state:



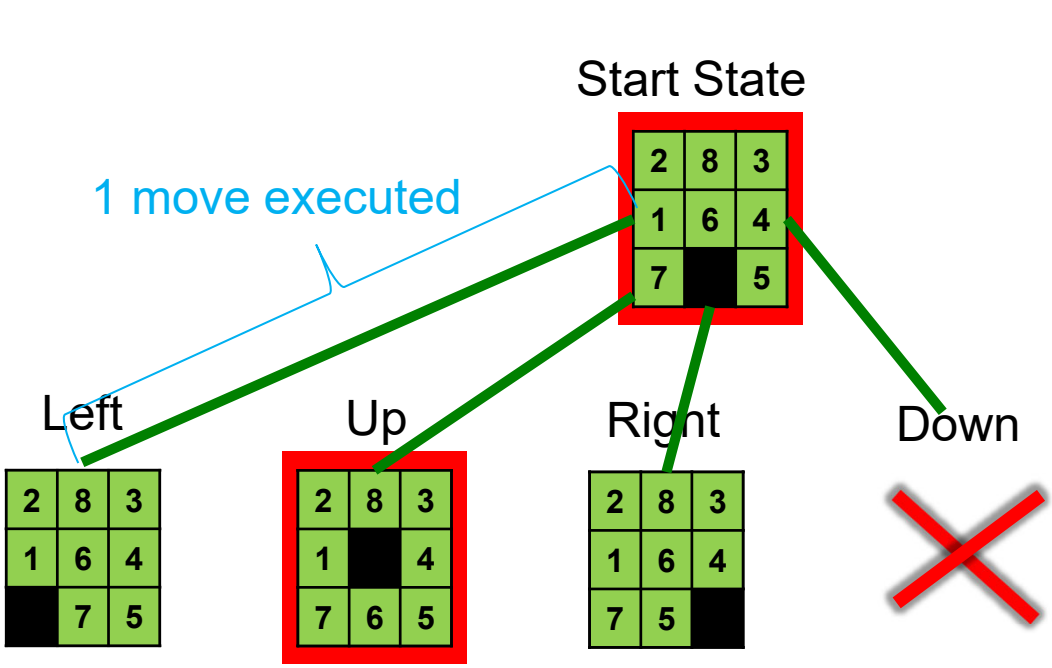
Goal State

1	2	3
8		4
7	6	5

	Left	Up	Right	Down
Heuristic value:	6	4	6	Sum of Manhattan Distance
+				+
g-value:	1	1	1	Number of moves made
F-cost	**7**	**5**	**7**	Estimate of the total number of moves required to reach the goal state

menu

Heuristic Search



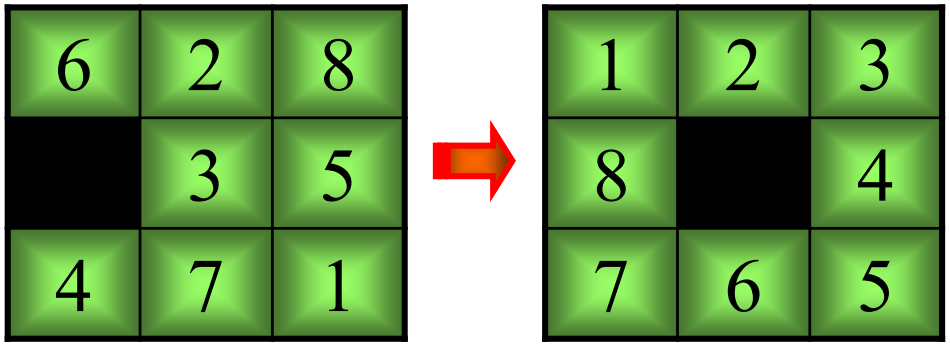
	Left	Up	Right	Down
	6	4	6	Sum of Manhattan Distance
	+ 1	+ 1	+ 1	+ Number of moves made
F-cost	7	5	7	Estimate of the total number of moves required to reach the goal state (**f-cost**)

minimum number of moves!

menu

Admissible Heuristics

8 Puzzle: Move tiles to reach goal. Think of a move as **moving an "empty" tile**.



S **G**

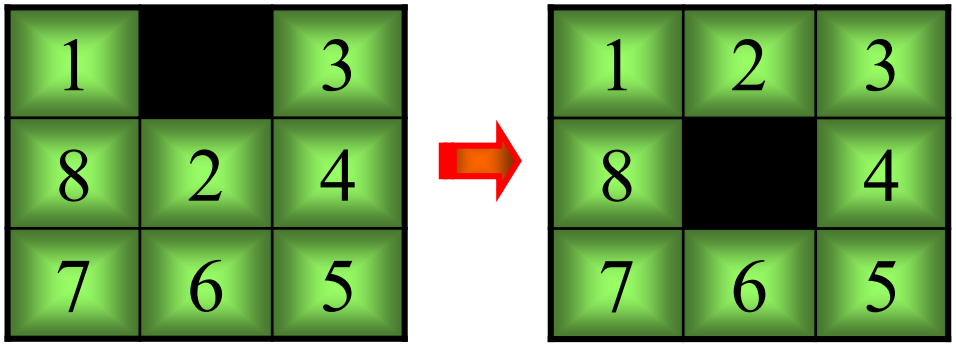
Alternative underestimates of "distance" (number of moves) to goal:

Number of misplaced tiles (7 in the example above).

Admissible Heuristics

8 Puzzle: Move tiles to reach goal. Think of a move as moving an "empty" tile.

Number of **misplaced tiles**



S **G**

Are we supposed to include the blank tile when counting the number of misplaced tiles?

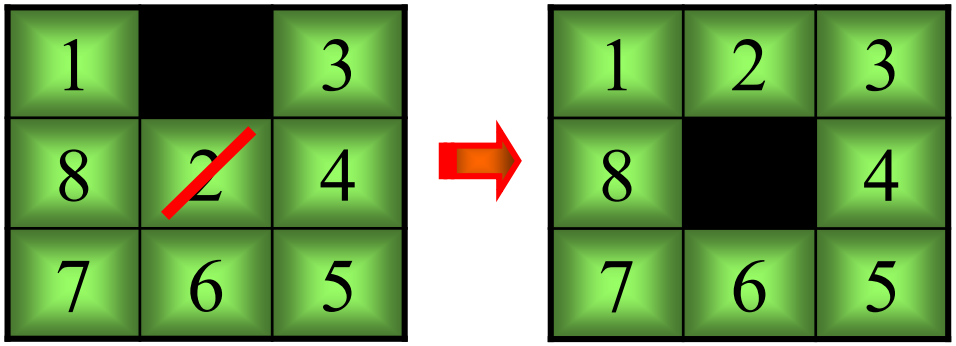
What will be the consequence if we include it? Examine the case above.

Number of misplaced tiles:

Admissible Heuristics

8 Puzzle: Move tiles to reach goal. Think of a move as moving an "empty" tile.

Number of **misplaced tiles**



S **G**

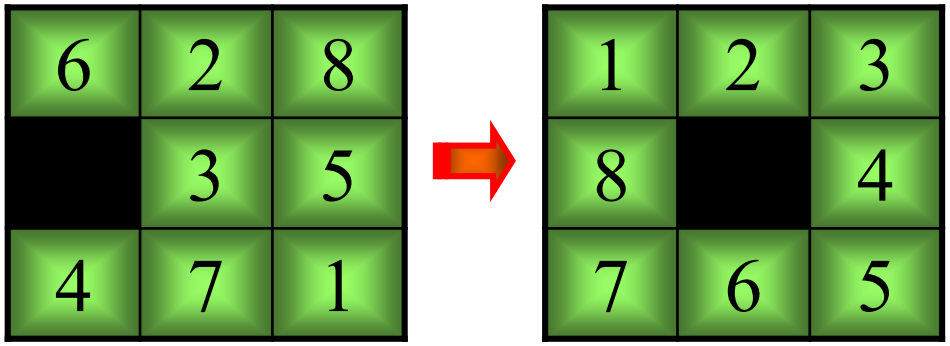
Are we supposed to include the blank tile when counting the number of misplaced tiles? **No.**

What will be the consequence if we include it? **the heuristic will not be admissible.**

Number of misplaced tiles: **1**

Admissible Heuristics

8 Puzzle: Move tiles to reach goal. Think of a move as moving an "empty" tile.



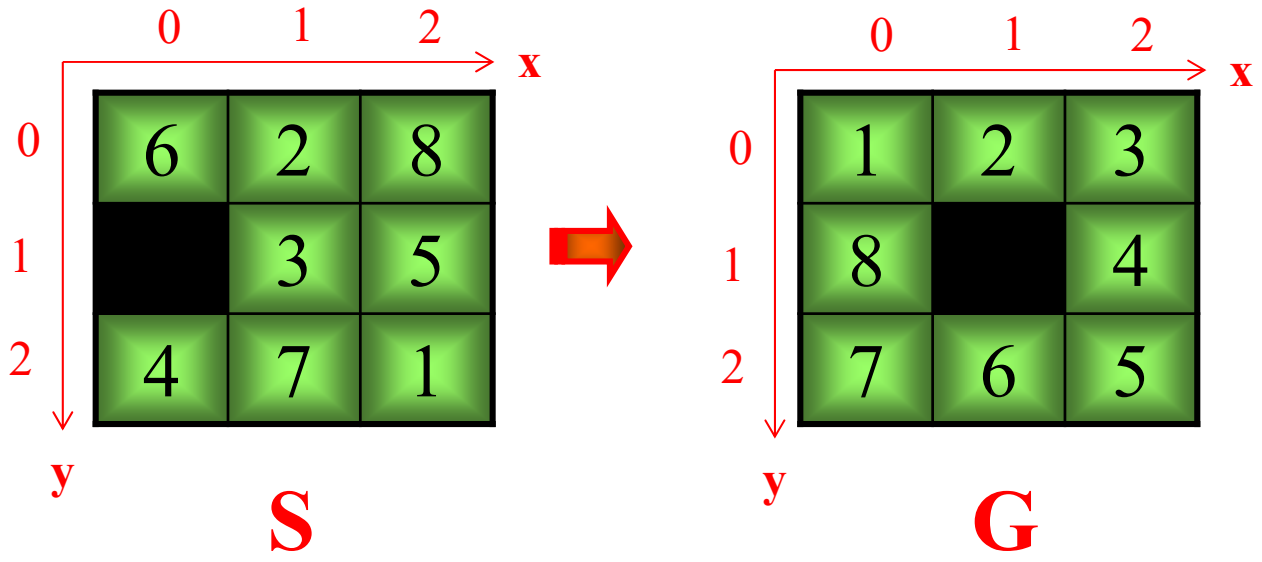
Alternative underestimates of "distance" (number of moves) to goal:

1. Number of **misplaced tiles** (7 in the example above).

2. Sum of **Manhattan Distance** of tile to its goal location (17 in example above).
   Manhattan Distance between (x_1, y_1) and (x_2, y_2) is $|x_1 - x_2| + |y_1 - y_2|$.
   Each move can only decrease the distance of exactly one tile.

The second of these is much better at predicting actual number of moves.

Admissible Heuristics



Note: Do not include the blank tile in the calculations.

Manhattan Distance between (x_1, y_1) and (x_2, y_2) = $|x_1 - x_2| + |y_1 - y_2|$

Absolute value

Number of Misplaced Tiles = **7**

Sum of Manhattan distance = **17**

